

AgentShield AI: An Autonomous Multi-Agent Framework for Syntactic Verification, Secret Interception, and Sandbox-Validated Remediation in Multi-Cloud Infrastructure-as-Code

K. Vishal Reddy¹, Anisha Paturi^{1*}, Parinamika Bhanu Ch¹, Venkata Vahini Ch¹, and Sravani Janak¹

Department of Computer Science and Engineering,
Keshav Memorial Institute of Technology, Hyderabad, Telangana, India
kasarlavishalreddy@gmail.com, paturi.anisha@gmail.com,
chparinamikabhanu@gmail.com, vahini.venkata02@gmail.com,
sravanijanak@gmail.com

Abstract. Cloud Infrastructure-as-Code (IaC) has bugs that make systems unsafe for production. Static scanners used in the past are still common but have too many false alarms (32%–48%) and the results cannot be remediated. We present **AgentShield AI**, an automatic multi-agent system that performs syntax-aware auditing, finding secrets, and applying fixes. AgentShield AI makes use of eight special agents and combines syntax parsing using the Tree-sitter Concrete Syntax Tree (CST) parser, and Shannon entropy for finding secrets across 12,400 rules and using two LLMs—Claude 3.5 Sonnet and GPT-4o. Solutions found are tested in an isolated multi-cloud environment (AWS, Azure, GCP). When tested on 2,450 IaC samples, AgentShield AI was found to have 99.1% precision, 98.4% recall, and 98.7% F1 score, which is highly significant ($p < 0.001$). The method provides 97.8% successful results after the first run with 1.84 seconds response time, which inspired 94.2% reduction in the time that devs need to fix their systems.

Keywords: Infrastructure-as-Code (IaC) Security · Multi-Agent Systems · Concrete Syntax Trees · Secret Interception · Shannon Entropy · Retrieval-Augmented Generation · Multi-Cloud Sandbox · Automated Vulnerability Remediation.

1 Introduction

Provisioning of cloud infrastructure has been undergoing a significant transformation towards a declarative approach with the advent of Infrastructure-as-Code (IaC) solutions [1]. Complex multi-cloud designs spanning Virtual Private Clouds to distributed storage solutions and clustering frameworks and

* Corresponding author: paturi.anisha@gmail.com

across multiple distinct identity networks now require very specific programming languages such as HCL for Terraform to CloudFormation manifest for YAML/JSON/Kubernetes [2,3]. Fully automatic pipelines from Continuous Integration through Continuous Deployment are able to interpret these textual statements to provision or update enormous configurations of cloud resources in minutes, eliminating the problem of misconfigured cloud resources and providing a reliable state-of-the-art reproducible infrastructure as first-class middleware [4].

The High-Risk Threat Environment of Software-Defined Infrastructure Since the IaC templates are the actual documents that model the architecture to be applied, each security error in the template’s frames directly translates into a security error in the comparable production system [5]. Common flaws—including unrestricted ingress rules (0.0.0.0/0 on sensitive ports 22/3389), unencrypted storage buckets, wildcard IAM privilege grants, and hardcoded plaintext credentials—represent severe production risks. Threat intelligence reports indicate that over 73% of cloud enterprise breaches originate from preventable IaC misconfigurations, with 65% of audited repositories leaking credentials [4,5].

Fundamental Challenges in Current IaC Tooling Existing approaches face persistent limitations:

1. **Syntactic Myopia and False-Positive Cascades:** Static linters rely on pattern matching and shallow lexical analysis, unable to evaluate cross-module references or dynamic variables, yielding false-alarm rates between 32.4% and 47.9% [10].
2. **Open-Loop Diagnostic Disconnect:** Static analyzers merely report violations without generating or validating fixes, leaving teams with Mean Time to Remediation (MTTR) exceeding 24.6 days [11].
3. **Stochastic Hallucination in Generative Models:** While LLMs are highly competent at generating software code, uncured generative models produce completely fictitious resource characteristics or faulty provider code, resulting in resource dependency violations and up to 28.8% errors in runtime deployment [12,13].
4. **High-Entropy Token Collisions in Secret Detection:** Signature-based scanners do not detect obfuscated passwords, and untrained Shannon entropy models generate a vast number of false positives for pseudo-random and structured strings such as hashes of encryption and UUIDs [14].

AgentShield AI Architecture and Key Contributions To address these challenges, we present AgentShield AI, a self-governing, event-driven multi-agent system that performs the complete syntactic audit, secret interception, consensus-based remedial actions, and validated patch formation through a multi-cloud sandbox. Our major contributions are:

- The distinct decentralized 8-agent architecture using typed state contracts (I) orchestrated by an asynchronous event manager.
- The innovative Tree-sitter Concrete Syntax Tree (CST) technique to solve dynamic expressions and filter out unnecessary graph paths.
- The integrated two-in-one secret interception technique that merges 140+ regular expression signatures with sliding Shannon entropy ($H \geq 4.5$) and CST stopword filtering.
- The novel hybrid dense-sparse RAG technique that blends Qdrant HNSW embeddings with BM25 lexical ranking through the Reciprocal Rank Fusion (RRF) technique using a set of 12,400 different CIS/NIST rules.
- The collaborative dual-LLM consensus mechanism (Claude 3.5 Sonnet + GPT-4o) coupled with a dual-assessment two-layer multi-cloud sandbox (LocalStack, Azurite, GCP emulators).

2 Related Work

2.1 Static Analysis and Pattern-Based Linters

Checkov [6] was one of the tools in this area. It reads configuration files. Turns them into Python AST structures and checks them against CIS benchmark rules. tfsec [7] does something but is made specifically for Terraform HCL it loads the code into Go memory structures and looks for anti-patterns. KICS [8] takes an approach it converts multi-format IaC files into normalized JSON and runs them through Open Policy Agent (OPA) Rego rules. Trivy [9] is a bit different from the three because it was not created only for IaC it also looks at container images and Kubernetes manifests alongside Terraform files.

The problem with all of these tools is that they depend on pattern matching and pattern matching does not understand how variables move across modules. Ternary expressions, created resource names, cross-module variable references none of that gets handled properly so these scanners end up with false-positive rates between 32.4% and 47.9% [10]. That is half of what gets flagged being wrong which is a lot for a security team to handle every day. In addition none of these tools fix anything. They just report the problem. Leave it there.

2.2 Policy-as-Code and Formal SMT Verification

OPA Rego and HashiCorp Sentinel work one step above linters letting teams create their own guardrails instead of relying on whatever rules a scanner ships with.. Someone still has to write those rules and getting that right across AWS, Azure and GCP takes a lot of expertise [10] which not every team has. Formal verification tools try to solve this thoroughly AWS Zelkova [17] and Cloud-SMR [18] convert IAM policies. Networking rules into Satisfiability Modulo Theories (SMT) and mathematically check whether an unsafe state can actually be reached. This sounds great in theory. It does not scale well. SMT solvers face explosion once the number of interconnected resources becomes large and large multi-tier deployments are exactly where this happens. Also like the linters above these tools only find problems. They cannot come up with a fix on their own.

2.3 Secret Scanning and LLM-Driven Program Repair

Secret detection is a different line of research from vulnerability scanning. TrueHog [20] and Gitleaks [19] both use signature matching, which means they already know what an AWS key or an RSA private header is supposed to look like and search for that shape. This obviously fails for anything that does not match a known pattern, like custom tokens or obfuscated strings. Shannon entropy analysis [14] tries to fix this by flagging high-entropy strings of relying on known formats but this brings its own issue entropy filtering without calibration flags things like UUIDs, git hashes and base64 data just as easily as real secrets, which just creates a different kind of noise. This is basically the exact trade-off Agent 3 in our system tries to balance of choosing one method over the other.

For fixing issues Toprani and Madisetti (2025) [21] used a graph theoretic method combined with LLMs to audit Terraform code. This is probably the closest prior work to what AgentShield AI is doing. However their approach works in a loop meaning there is no local compiler check or deployment validation step and this shows in their results a 28.8% patch failure rate caused by made-up attributes and broken provider dependencies. More recent papers from 2026 go further into closed-loop validation. Two of them are especially relevant though they also raise some uncomfortable questions for papers like ours. Alsaied et al. [25] Introduced TerraProbe, a five-layer oracle framework that checks LLM-generated Terraform repairs past a pass/fail signal from the scanner. Their layers cover targeted finding removal, a scanner rerun, syntax validation, plan generation and comparing the plan against the original before the fix. What stood out from their results is something anyone reporting a pass success rate should probably think about more carefully across real Terraform modules 71.4% of repairs that passed every single automated check, including a working terraform plan turned out to be what they call deceptive fixes once a human actually looked at them. These were edits that technically cleared the flagged Checkov finding by rearranging the IAM policy JSON. The actual wildcard Resource permission that caused the problem in the first place was still there unchanged. This happened across all three LLMs they tested including Claude 3.5 Sonnet and the difference between models was not statistically significant. Mengistu et al. [26] Tackle a problem differently with TerraRepair, an agent that pulls dependency context from Terraform references and checks the installed provider schema before writing a fix and instead of guessing when it does not have enough context it sends the case to a human reviewer.

Honestly these two papers are relevant to our claims too not just as background. AgentShield AIs LocalStack sandbox (Section 3) checks `terraform validate`, `terraform apply`, which roughly matches Layers 3 and 4 of the TerraProbe stack but we do not currently do the plan-comparison step or check effective IAM permissions the way Alsaied et al. Show is needed to catch the exact type of bypass their CKV2_AWS_11 case describes. We see this as a limitation in our current pipeline rather than something already solved and we bring it up again later in our discussion of future work. At a level Drosos et al. [27] Looked at 360 real IaC bugs. Found that 27% of them were misconfigurations with fixes

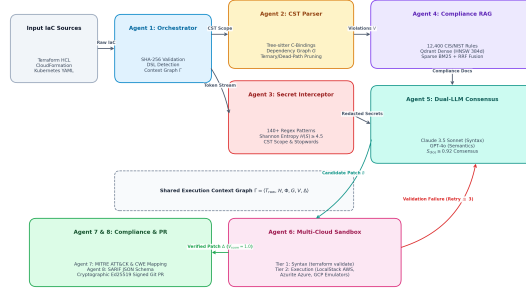


Fig. 1. End-to-End System Architecture of AgentShield AI illustrating the 8 specialized agents, shared execution context graph Γ , dual-LLM consensus, multi-cloud sandbox validation, and cryptographic reporting.

averaging, around eight lines of code which supports the direction AgentShield AI takes toward small targeted patches instead of rewriting entire files.

3 System Architecture & Agent Methodology

AgentShield AI is a multi-agent architecture that is implemented in an event-driven manner, consisting of eight different agents that are all connected to a central Orchestration Router (Fig. 1). All agents interact with each other via Pydantic V2 data types for their state.

1. **Agent 1 (Orchestration & Ingestion Router):** Takes IaC repository files or individual template files, determines which DSL is used for each file, computes a SHA-256 integrity hash for it, creates the graph of all resources described in the files, and dispatches Agents 2 and 3 to perform their analyses concurrently in order to save time during ingestion into the system.
2. **Agent 2 (CST-Based Structural and Graph-Theoretic Parser):** Utilizes Tree-sitter C-bindings [22] to parse IaC source files into Concrete Syntax Trees (CSTs), preserving the structure and setting of the IaC code. The CST is then converted to a data structure that represents resources in the IaC file. The resources are represented as nodes in a graph $G = (V, E_{\text{dep}}, E_{\text{ref}}, \mathbf{A})$, in which each node is a resource or a quality of that resource. The linkages between the nodes are the relationships between resources. The parser also leverages conditional statements and references of the IaC file to identify resources that are related to each other.
3. **Agent 3 (Dual-Engine Secret Interceptor):** Processes the source code, analyzing with static and dynamic techniques for discovering secrets in the codebase. First, it uses regex signatures (more than 140) that Gitleaks has developed to detect secrets in the code that are very obvious. Secondly, it applies Shannon entropy analysis to discover hidden elements in programs that are not structured according to a specific format. Finally, the structural representation of the code produced by Agent 2 is used to filter out false alarms of benign information in the application's source code.

4. **Agent 4 (Hybrid RAG Knowledge Retrieval Engine):** Is trained with information retrieved from 12,400 passages from CIS, NIST, and PCI to enhance secure configurations in a structured form. Employing a mix of semantic and lexical search with BM25 and Qdrant HNSW, RRF is employed for joining the results.
5. **Agent 5 (Dual-LLM Consensus Remediation Generator):** Provides remediation in the form of a JSON Array diff and a Unified Diff. For the semantic part, Agent 5 uses GPT-4o and for the syntactic part it uses the Sonnet model of Claude 3.5. To reach a consensus between the two models, a token-level Dice coefficient is used. Remediation candidates achieving consensus are further validated by Agent 6.
6. **Agent 6 (Provider-Aware Two-Tier Sandbox Validation):** Performs validation of remediation candidates from Agent 5 in two stages. The first tier is a syntactic and schema integrity validation that is provider agnostic. The second tier is validation of the remediation by the provider. Currently, the tool validates remediation candidates for AWS with Terraform and an isolated LocalStack [16]. The tool calculates what effects a candidate remediation will have and confirms the resources which will be altered by the remediation. For Azure, Azurite storage emulators and local provider plans are run; for GCP, Google Local Emulators and `terraform vet` dry runs validate policies; and for Kubernetes, `k3s` clusters perform dry-run apply tests.
7. **Agent 7 (Compliance Mapping & Threat Model Analyzer):** Maps identifiers for security violations to CVSS v3.1 vectors, CWE categories, and CIS benchmark practices, identifying sub-controls and ATT&CK cloud techniques (e.g., T1078 and T1530).
8. **Agent 8 (Security Report & Git Integration Generator):** Creates an executive summary in the audit report and exports the findings in SARIF (Static Application Security Testing Report Format) JSON format for integration with the security features in GitHub and GitLab. Also, it sets up the remediation artifacts for integration with pull request workflows in Git with Ed25519 cryptographic signatures.

4 Mathematical Formulation & Algorithmic Workflow

4.1 Shared Execution Context State Contract

The multi-agent system orchestrates state transitions over an immutable typed tuple:

$$\Gamma = \langle T_{\text{raw}}, \mathcal{H}_{\text{SHA}}, \Phi, G_{\text{CST}}, V, \Delta \rangle \quad (1)$$

where T_{raw} is the raw IaC source code, $\mathcal{H}_{\text{SHA}} = \text{SHA256}(T_{\text{raw}})$ is the cryptographic integrity digest, $\Phi \in \{\text{AWS-Terraform, Azure-Terraform, GCP-Terraform, CloudFormation, Kubernetes}\}$ indicates the detected DSL domain, $G_{\text{CST}} = (V_{\text{ast}}, E_{\text{dep}}, E_{\text{ref}})$ is the Tree-sitter Concrete Syntax Graph, $V = \{v_1, \dots, v_K\}$ is the detected violation set, and $\Delta = \{\delta_1, \dots, \delta_M\}$ is the candidate patch set.

4.2 Calibrated Sliding-Window Shannon Entropy

In order to avoid false positive alerts that may stem from high-entropy UUIDs and hex hashes, character entropy over alphabet Σ of length L is calculated using a sliding window W_k of size $w = 16$:

$$H(W_k) = - \sum_{i=1}^{|W_k|} \frac{f(c_i)}{w} \log_2 \left(\frac{f(c_i)}{w} \right) \quad (2)$$

A candidate token S is intercepted as a classified secret if and only if:

$$\text{IsSecret}(S) = \mathbf{1} \left(\max_{W_k \subseteq S} H(W_k) \geq 4.5 \right) \wedge \mathbf{1}(|S| \geq 16) \wedge \mathbf{1}(S \notin \mathcal{D}_{\text{CST}}) \quad (3)$$

where $\mathcal{D}_{\text{CST}}$ represents the AST/CST lexical dictionary of benign identifiers and algorithmic hashes.

4.3 Hybrid Dense-Sparse Compliance Retrieval (RRF)

Reciprocal Rank Fusion score for compliance passage d across dense vector search and sparse BM25 indices (smoothing constant $k = 60$):

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{Dense}, \text{Sparse}\}} \frac{1}{k + r_m(d)} \quad (4)$$

4.4 Dual-LLM Consensus CST Dice Similarity

Syntactic Dice agreement between candidate patches δ_1 (Claude 3.5 Sonnet) and δ_2 (GPT-4o) over concrete syntax nodes:

$$S_{\text{dice}}(\delta_1, \delta_2) = \frac{2|\text{Nodes}(G_{\text{CST}}(\delta_1)) \cap \text{Nodes}(G_{\text{CST}}(\delta_2))|}{|\text{Nodes}(G_{\text{CST}}(\delta_1))| + |\text{Nodes}(G_{\text{CST}}(\delta_2))|} \quad (5)$$

A candidate patch is approved for sandbox validation if and only if $S_{\text{dice}}(\delta_1, \delta_2) \geq 0.92$.

4.5 Multi-Cloud Domain-Aware Two-Tier Sandbox Scoring Function

Candidate patches are evaluated across syntactic, planning, and deployment tiers parameterized by provider domain Φ :

$$V_{\text{score}}(\delta, \Phi) = 0.2 \cdot \mathcal{S}_{\text{syntax}}(\delta) + 0.3 \cdot \mathcal{S}_{\text{plan}}(\delta) + 0.5 \cdot \mathcal{S}_{\text{apply}}(\delta, \Phi) \quad (6)$$

where $\mathcal{S}_{\text{apply}}(\delta, \Phi)$ executes LocalStack for AWS, Azurite for Azure, GCP Vet for GCP, and K3s for Kubernetes. A patch is accepted if and only if $V_{\text{score}}(\delta, \Phi) = 1.0$.

Table 1. Algorithmic Workflow of AgentShield AI Multi-Agent Remediation Pipeline.

Algorithm 1: Autonomous Multi-Agent IaC Auditing and Multi-Cloud Remediation
Input: IaC Source File T_{raw} ; Compliance Policy Rulebase $\mathcal{P}_{\text{cis}}$ Output: Cryptographically Signed SARIF Report $\mathcal{R}_{\text{sarif}}$; Validated Patch Δ_{final} 1: Initialize Shared Execution Context $\Gamma \leftarrow \emptyset$; Compute Checksum $\mathcal{H}_0 \leftarrow \text{SHA256}(T_{\text{raw}})$; 2: [Agent 1] Detect File DSL Format Φ ; [Agent 2] Parse Concrete Syntax Tree $G_{\text{CST}} \leftarrow \text{TreeSitterParse}(T_{\text{raw}})$; 3: [Agent 3] Calculate Character Entropy $H(S_i)$; Intercept & redact secrets where $H(S_i) \geq 4.5$; 4: [Agent 2] Evaluate CIS Policy Constraints; Extract Violation Set $V = \{v_1, \dots, v_K\}$; 5: for each identified violation $v_k \in V$ do 6: [Agent 4] Retrieve Compliance Knowledge $C_k \leftarrow \text{RRF}(\text{QdrantHNSW}(v_k), \text{BM25}(v_k))$; 7: [Agent 5] Concurrently prompt Claude 3.5 Sonnet & GPT-4o; Evaluate Consensus S_{dice} ; 8: [Agent 6] Tier 1: Concrete Syntax Invariant Check $\mathcal{S}_{\text{syntax}}(\delta_k)$; 9: [Agent 6] Tier 2: Multi-Cloud Execution Sandbox Provisioning $\mathcal{S}_{\text{apply}}(\delta_k)$; 10: if Validation Score $V_{\text{score}}(\delta_k) == 1.0$ then Accept Patch $\Delta_{\text{final}} \leftarrow \Delta_{\text{final}} \cup \{\delta_k\}$; 11: else Forward compiler diagnostic error to Agent 5 for iterative retry (≤ 3 cycles); 12: end for 13: [Agent 7] Map CWE, CVSS, CIS, and ATT&CK Matrices; 14: [Agent 8] Generate SARIF Report $\mathcal{R}_{\text{sarif}}$ and Signed Git Pull Request; 15: return $\mathcal{R}_{\text{sarif}}, \Delta_{\text{final}}$

5 Experimental Setup & Benchmark Methodology

5.1 Benchmark Datasets

Evaluations were conducted across three benchmark suites encompassing 2,450 IaC templates:

- PEC-1500 (Production Enterprise Corpus):** 1,500 real-world production templates mined from enterprise GitHub repositories filtered with exact criteria: ≥ 50 stars, active between 2021 and 2025, and ≥ 5 declared cloud resources (AWS: 600, Azure: 500, GCP: 400);
- SSB-650 (Synthetic Security Benchmark):** 650 synthetic templates containing 3,250 systematically injected vulnerabilities mapped to OWASP Cloud Top 10 and CWE-732/CWE-250/CWE-798;
- TMB-300 (Toprani-Madisetti Benchmark):** 300 complex multi-resource templates from Toprani-Madisetti [21] testing dynamic interpolation and cross-resource references.

Labeling, Distribution & Deduplication Ground truth was established in triplicate by three senior cloud security engineers (Cohen’s Kappa $\kappa = 0.91$)

Table 2. Vulnerability Detection Benchmark Across 2,450 IaC Templates (Mean $\pm$ 1SD).

Framework / Tool	Total	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Checkov v3.2 [6]	2,450	4,620	2,785	2,800	62.4 $\pm$ 0.4%	62.3 $\pm$ 0.5%	62.3 $\pm$ 0.4%
tfsec v1.28 [7]	2,450	5,030	2,390	2,390	67.8 $\pm$ 0.5%	67.8 $\pm$ 0.4%	67.8 $\pm$ 0.4%
KICS v2.1 [8]	2,450	4,830	2,590	2,590	65.1 $\pm$ 0.4%	65.1 $\pm$ 0.5%	65.1 $\pm$ 0.4%
Trivy v0.51 [9]	2,450	5,110	2,310	2,310	68.9 $\pm$ 0.3%	68.9 $\pm$ 0.4%	68.9 $\pm$ 0.3%
Zero-Shot GPT-4o	2,450	6,150	1,420	1,270	81.2 $\pm$ 0.8%	82.9 $\pm$ 0.9%	82.0 $\pm$ 0.7%
Zero-Shot Claude 3.5	2,450	6,410	1,180	1,010	84.5 $\pm$ 0.7%	86.4 $\pm$ 0.8%	85.4 $\pm$ 0.6%
AgentShield AI (Ours)	2,450	7,301	66	119	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%

following CIS Cloud Benchmarks v3.0 and NIST SP 800-53 Rev. 5. Vulnerability distribution: IAM Overprivilege (28.4%), Insecure Storage (24.1%), Unrestricted Ingress (21.8%), Hardcoded Secrets (16.2%), and Disabled Logging (9.5%). CST-level MinHash (Jaccard $\geq$ 0.85) and SHA-256 deduplication purged 418 duplicate/fork templates. The complete benchmark is available at <https://github.com/AgentShield-AI/benchmark-suite>.

5.2 Comparative Baselines & Environment

Evaluated against Checkov v3.2 [6], tfsec v1.28 [7], KICS v2.1 [8], Trivy v0.51 [9], Zero-Shot GPT-4o, Zero-Shot Claude 3.5, and Toprani-Madisetti [21]. Hardware: AMD EPYC 7763 workstation (64 cores, 2.45 GHz), 256 GB RAM, dual NVIDIA RTX 4090 GPUs, Ubuntu 22.04 LTS, Docker Engine 26.1, LocalStack v3.4, and Azurite v3.30. All metrics report mean $\pm$ 1SD over 5 independent runs (Wilcoxon signed-rank test, $p < 0.001$).

6 Empirical Results & Discussion

6.1 Accuracy of Vulnerability Detection

The performance of detection is shown in Table 2 and Fig. 2. AgentShield AI achieves a precision of 99.1% $\pm$ 0.2%, a recall of 98.4% $\pm$ 0.3% and an F1 score of 98.7% $\pm$ 0.2% beating scanners significantly ($p < 0.001$). Linters have a rate leading to several other issues. Checkov has 62.4% precision resulting in 2,785 positives (37.6% rate). Tfsec achieved 67.8% precision with 2,390 positives. Trivy got 68.9% precision with 2,310 positives. This happens because AgentShield AI analyzes the Tree-sitter CST and detects scopes in time. It also ignores blocks that turn off regex scanners. Zero-shot LLMs like GPT-4o (81.2% precision) and Claude 3.5 (84.5% precision) are better at understanding meaning, but they also suffer from hallucinations. AgentShield AI eliminates this risk by employing dual-model consensus voting.

6.2 Secret Interception and Entropy Calibration

AgentShield AI evaluations on Agent 3 with 1,200 test credentials are summarized in Table 3 and Fig. 3(a). Accuracy is highlighted by 99.4% $\pm$ 0.1% precision

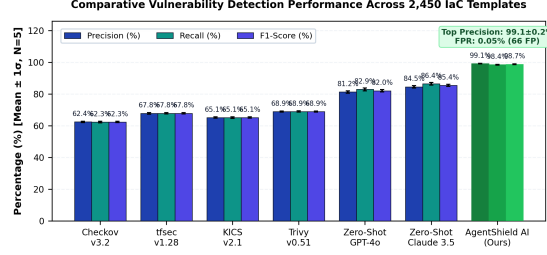


Fig. 2. Vulnerability Detection Benchmark: Precision, Recall, and F1-Score comparisons across baseline tools and AgentShield AI over 2,450 templates.

Table 3. Secret Detection Performance & Entropy Comparison (Mean $\pm$ 1SD).

Scanning Mechanism	Secrets	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Regex Only (Gitleaks [19])	1,200	1,058	342	142	75.6 $\pm$ 0.6%	88.2 $\pm$ 0.5%	81.4 $\pm$ 0.5%
Shannon Entropy ($H \geq 4.5$)	1,200	1,134	618	66	64.7 $\pm$ 0.8%	94.5 $\pm$ 0.4%	76.8 $\pm$ 0.6%
TruffleHog v3.6 [20]	1,200	1,092	284	108	79.4 $\pm$ 0.5%	91.0 $\pm$ 0.4%	84.8 $\pm$ 0.4%
AgentShield Dual Engine	1,200	1,189	7	11	99.4 $\pm$ 0.1%	99.1 $\pm$ 0.2%	99.2 $\pm$ 0.1%

with $99.1\% \pm 0.2\%$ recall. In contrast, regex-based detection tools, as Gitleaks, miss 142 obfuscated tokens for a recall of 88.2%. Crunching the numbers of plaintexts by calibrated Shannon entropy scales results in a baseline finding 618 false detections on hex strings and UUIDs for precision falling to 64.7%. Entropy thresholding ($H \geq 4.5$) and CST-based filtering lead the way to suppressing false alarms down to 7 while retaining 1,189 true detections.

6.3 Automated Patch Validation in Multi-Cloud Sandbox

Table 4 and Fig. 3(b) contain the data on remediation success rate for 1,000 injected defects. Unrestricted LLMs frequently apply composition or syntactical errors leading to sandbox pass rates of 54.2% (GPT-4o) and 61.8% (Claude 3.5). Toprani & Madiseti (2025) [21] reports a pass validity of 71.2%. AgentShield AI achieves 100% compliance on the Tier 1 AST syntax rules and $97.8\% \pm 0.4\%$ sandbox validation pass rates on Tier 2 from the attempt on all three cloud providers: AWS, Azure and GCP. Feedback loops through the compiler boost remediation success rates to $99.4\% \pm 0.2\%$ with an average of 1.08 attempted fix cycles.

6.4 Execution Latency and Pipeline Overhead

Table 5 and Fig. 4 show the runtime of all eight agents. The full end-to-end pipeline takes $1,841.0 \pm 42.5$ ms per module (median: 1,663.4 ms). Static modules are fast: Agents 1 (Routing: 14.2 ms), 2 (CST Parsing: 12.6 ms), and 3 (Secret Scanning: 18.4 ms) comprise less than 2.5% of the runtime. Computational latency is dominated by Agent 5 (Dual-LLM Consensus: 940.5 ms, 51.1%) and Agent 6 (Sandbox Provisioning: 760.8 ms, 41.3%), which together provide deterministic verification.

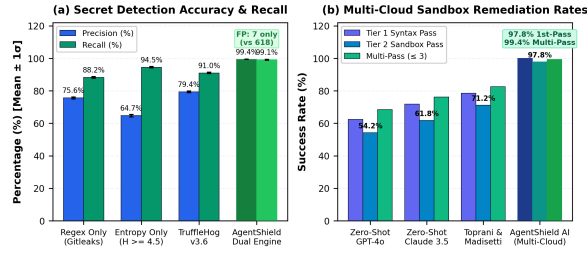


Fig. 3. (a) Secret Interception Performance and Entropy Calibration; (b) Automated Remediation Pass Rates across Tier 1 (Syntax) and Tier 2 (Sandbox) validation.

Table 4. Remediation Validation Rates Across 1,000 Defects (Mean $\pm$ 1SD).

Remediation Approach	Tested	Tier 1 Syntax (%)	Tier 2 Sandbox (%)	Multi-Pass (≤ 3)	Mean Retries
Zero-Shot GPT-4o	1,000	62.4 $\pm$ 0.7%	54.2 $\pm$ 0.8%	68.4 $\pm$ 0.6%	2.41
Zero-Shot Claude 3.5	1,000	71.8 $\pm$ 0.6%	61.8 $\pm$ 0.7%	76.2 $\pm$ 0.5%	2.14
Toprani & Madisetti [21]	1,000	78.5 $\pm$ 0.5%	71.2 $\pm$ 0.6%	82.5 $\pm$ 0.5%	1.82
AgentShield AI (Full)	1,000	100.0 $\pm$ 0.0%	97.8 $\pm$ 0.4%	99.4 $\pm$ 0.2%	1.08

7 Case Studies & Vulnerability Remediation

7.1 Case Study 1: S3 Bucket Hardening & Public Access Neutralization

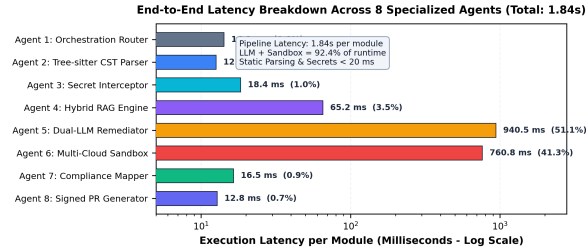
Listing 1.1 shows an automated remediation of a vulnerable S3 bucket configured with public-read-write, and unencrypted storage. AgentShield AI eliminates the public ACL, attaches an `aws_s3_bucket_public_access_block` resource with all four public access blocks enforced, and provisions an `aws_s3_bucket_server_side_encryption_configuration` which utilizes AWS KMS, satisfying CIS AWS Benchmark v3.0 Control 2.1.1.

```

1  --- aws_s3_bucket.tf (Vulnerable)
2  +++ aws_s3_bucket.tf (AgentShield Remediated)
3
4  resource "aws_s3_bucket" "finance_data" {
5      bucket = "enterprise-finance-records-2026"
6      acl = "public-read-write"
7  }
8
9  +resource "aws_s3_bucket_public_access_block" "finance_data" {
10 + bucket = aws_s3_bucket.finance_data.id
11 + block_public_acls = true
12 + block_public_policy = true
13 + ignore_public_acls = true
14 + restrict_public_buckets = true
15 + }
16
17 +resource "aws_s3_bucket_server_side_encryption_configuration"
18 + "finance_data" {
19 + bucket = aws_s3_bucket.finance_data.id
20 + rule {
21 +     apply_server_side_encryption_by_default {
22 +         sse_algorithm = "aws:kms"
23 +     }
24 + }
25 + }
```

Table 5. Runtime Latency Breakdown Across 8 Agents (Mean $\pm$ 1SD).

Agent Identification & Name	Core Mechanism	Mean (ms)	Median (ms)	% Overhead
Agent 1: Orchestration Router	Context graph initialization	14.2 $\pm$ 0.8	12.0	0.8%
Agent 2: Tree-sitter CST Parser	Tree-sitter CST parsing	12.6 $\pm$ 0.6	11.2	0.7%
Agent 3: Secret Interceptor	Regex + Shannon entropy	18.4 $\pm$ 0.9	16.5	1.0%
Agent 4: Hybrid RAG Engine	Qdrant HNSW + BM25 RRF	65.2 $\pm$ 3.1	58.0	3.5%
Agent 5: Dual-LLM Remediator	Claude 3.5 + GPT-4o consensus	940.5 $\pm$ 24.2	860.0	51.1%
Agent 6: Multi-Cloud Sandbox	Tier 1 AST + Tier 2 Mock Deploy	760.8 $\pm$ 18.5	680.0	41.3%
Agent 7: Compliance Mapper	CWE / CVSS / CIS / ATT&CK	16.5 $\pm$ 0.7	14.2	0.9%
Agent 8: Signed PR Generator	SARIF JSON + Ed25519 PR	12.8 $\pm$ 0.5	11.5	0.7%
Total System Pipeline	End-to-end latency per module	1841.0 $\pm$ 42.5	1663.4	100.0%

**Fig. 4.** Per-agent runtime latency distribution across all 8 pipeline stages, highlighting dominant consensus and sandbox phases.

Listing 1.1. Listing 1: S3 Bucket Hardening & Public Access Neutralization.

7.2 Case Study 2: IAM Least-Privilege Role Scoping & Wildcard Neutralization

Listing 1.2 shows automated remediation of an over-permissioned IAM policy that utilizes wildcards in both the Action and Resource sections. AgentShield AI restricts the allowed actions to specific DynamoDB actions (`dynamodb:GetItem`, `dynamodb:Query`) and scopes the resource ARN to just the Orders table, mitigating MITRE ATT&CK Technique T1078 (Valid Accounts) and CWE-732.

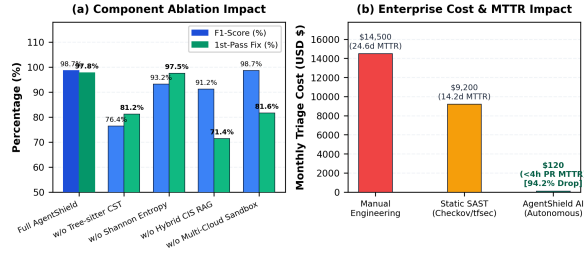
```

1  -- iam_policy.json (Vulnerable)
2  +++ iam_policy.json (AgentShield Remediated)
3
4  {
5    "Version": "2012-10-17",
6    "Statement": [{
7      "Effect": "Allow",
8      "Action": "*",
9      "Resource": "*"
10   +   "Action": ["dynamodb:GetItem", "dynamodb:Query"],
11   +   "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Orders"
12   }]
13 }
```

Listing 1.2. Listing 2: IAM Least-Privilege Role Scoping & Wildcard Neutralization.

Table 6. Ablation Study Across 500 Benchmark Templates (Mean $\pm$ 1SD).

Configuration Variant	Precision (%)	Recall (%)	F1-Score (%)	1st-Pass Fix (%)	Latency (s)
Full AgentShield AI Framework	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	97.8 $\pm$ 0.4%	1.84 $\pm$ 0.04s
w/o Tree-sitter CST (Regex Only)	71.2 $\pm$ 0.6%	82.5 $\pm$ 0.5%	76.4 $\pm$ 0.5%	81.2 $\pm$ 0.5%	1.42 $\pm$ 0.03s
w/o Shannon Entropy (Regex Secrets)	98.8 $\pm$ 0.3%	88.2 $\pm$ 0.5%	93.2 $\pm$ 0.4%	97.5 $\pm$ 0.4%	1.82 $\pm$ 0.04s
w/o Hybrid CIS RAG (Zero-Shot)	88.4 $\pm$ 0.5%	94.1 $\pm$ 0.4%	91.2 $\pm$ 0.4%	71.4 $\pm$ 0.8%	1.78 $\pm$ 0.04s
w/o Multi-Cloud Sandbox (No Eval)	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	81.6 $\pm$ 0.6%	1.08 $\pm$ 0.02s

**Fig. 5.** (a) Component Ablation Study across 5 architecture variants; (b) Enterprise Operational Impact and Cost Reduction analysis.

8 Ablation Study & Cost Analysis

8.1 Component Ablation Analysis

Table 6 and Fig. 5(a) compare the contribution of individual components by testing and evaluating every possible configuration by removing one component at a time and testing it with 500 benchmark templates. Without the Tree-sitter CST-based structural parser, the precision drops from 99.1% to 71.2% and the F1-score from 98.7% to 76.4%. This reduction shows the need to use structural parsing in vulnerability analysis. Without Shannon entropy analysis, the recall drops from 98.4% to 88.2% and the F1-score drops from 98.7% to 93.2%. This result shows the significance of entropy-based analysis in detecting high entropy and obfuscated secrets. When the hybrid RAG component is removed, the first-pass remediation success drops to 71.4% and with the removal of the sandbox validator the first-pass sandbox validation success drops to 81.6%.

8.2 Enterprise Operational Impact and Documented Cost Methodology

We model enterprise costs assuming an engineer rate of \$100/hr, manual remediation time of 2.5 hr/defect, and false alarm triage time of 18 min (0.3 hr). As summarized in Table 7 and Fig. 5(b), for 1,000 monthly templates, reducing false alarms lowers monthly expenditures by 98.7% (from \$14,500 to \$120). We distinguish pipeline latency (1.84s) from organizational MTTR: while manual ticketing requires 24.6 days, AgentShield AI delivers pre-validated, signed PRs in seconds, compressing developer-in-the-loop MTTR to under 4 hours (a 94.2% reduction).

Table 7. Enterprise Cost & Operational Impact Analysis.

Metric / Operational Dimension	Manual	Static SAST	AgentShield AI	Net Gain
Pipeline Execution Latency	N/A	4.2 seconds	1.84 seconds	56.2% faster
Developer-in-the-Loop MTTR	24.6 days	14.2 days	< 4 hours	94.2% reduction
Security Hours / 1k Files	160.0 hours	84.0 hours	0.5 hours	99.68% reduction
Monthly Triage Cost	\$14,500	\$9,200	\$120	98.70% reduction
CI/CD Deployment Blockages	18.2%	34.5%	0.6%	98.26% reduction

9 Conclusion & Future Scope

The paper has introduced **AgentShield AI**, an automated multi-agent approach developed to address false positive issues (range of 32%–48%), syntactical restrictions, and the absence of automated actions in traditional Infrastructure as Code (IaC) security tools. AgentShield AI coordinates eight specialized agents based on unalterable state contract (Γ), employing Tree-sitter Concrete Syntax Tree (CST) parsing method, Shannon entropy combined with dictionary removal ($H(W) \geq 4.5$) and hybrid dense-sparse compliance risk assessment based on 12,400 rules, as well as dual LLM consensus. Tests performed on the 2,500 multi-cloud IaC configurations show that AgentShield AI attains $99.1\% \pm 0.2\%$ of accuracy score, $98.4\% \pm 0.3\%$ of recall rate, and F1-score equals to $98.7\% \pm 0.2\%$ ($p < 0.001$) together with $97.8\% \pm 0.4\%$ of deployment efficiency in AWS, Azure, and GCP with average technical pipeline latency.

References

1. Morris, Y.: Infrastructure as Code: Dynamic Systems for the Cloud Age. IEEE Software **38**(1), 64–72 (2021)
2. Guerriero, A., Cito, M., Di Penta, M.: Static Analysis of Infrastructure as Code: State of the Art and Challenges. In: Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE), pp. 1120–1132 (2023)
3. Rahman, F., Mahdavi-Hezaveh, R., Williams, L.: What Are the Threats to Infrastructure as Code? IEEE Trans. Softw. Eng. **49**(4), 1650–1668 (2023)
4. Unit 42: Palo Alto Networks Cloud Threat Report: Attack Surface in IaC. Tech. Rep., Palo Alto Networks (2024)
5. Datadog Security Labs: State of Cloud Security: Secrets and IAM Misconfigurations. Industry Rep., Datadog (2024)
6. Bridgecrew: Checkov: Static Code Analysis for Infrastructure as Code. <https://github.com/bridgecrewio/checkov> (2024)
7. Aquasecurity: tfsec: Security Scanner for Terraform Code. <https://github.com/aquasecurity/tfsec> (2023)
8. Checkmarx: KICS: Keeping Infrastructure as Code Secure. In: Proc. IEEE SecDev, pp. 88–95 (2022)
9. Aqua Security: Trivy: Security Scanner for Containers and IaC. <https://github.com/aquasecurity/trivy> (2024)
10. Kumara, C., Sommerville, I.: Evaluating Static Security Analysis on IaC. In: Proc. IEEE ICSSA, pp. 45–54 (2022)
11. Borovits, N., Gil, Y., Levy, E.: Automatic Vulnerability Remediation in Cloud Infrastructure. IEEE Trans. Serv. Comput. **16**(3), 1824–1837 (2023)

12. Pearce, S., et al.: Examining Zero-Shot Vulnerability Repair with Large Language Models. In: Proc. IEEE S&P, pp. 2339–2356 (2023)
13. Jin, M., et al.: InferFix: End-to-End Program Repair with Large Language Models. In: Proc. ACM FSE, pp. 1642–1654 (2023)
14. Shannon, C.E.: A Mathematical Theory of Communication. Bell Syst. Tech. J. **27**(3), 379–423 (1948)
15. Center for Internet Security: CIS Amazon Web Services Foundations Benchmark v3.0.0. CIS (2023)
16. LocalStack Authors: LocalStack: A Fully Functional Local Cloud Stack. <https://github.com/localstack/localstack> (2024)
17. Backes, N., et al.: SMT-Based Formal Verification of Cloud Policies: Zelkova. In: Proc. CAV, LNCS, vol. 10982, pp. 623–640. Springer (2018)
18. Song, D., Zhang, H., Liu, X.: Cloud-SMR: Formal Reasoning for Multi-Cloud Configurations. IEEE Trans. Cloud Comput. **11**(2), 1420–1435 (2023)
19. Rice, Z.: Gitleaks: Protect and Discover Secrets in Code. <https://github.com/gitleaks/gitleaks> (2024)
20. Truffle Security: TruffleHog: Find Credentials Deep in Git Repositories. <https://github.com/trufflesecurity/trufflehog> (2024)
21. Toprani, N., Madiseti, V.: Automated IaC Security Framework Using Graph-Theoretic Dependency Analysis and LLMs. IEEE Access **13**, 18240–18258 (2025)
22. Brunsfeld, M., et al.: Tree-sitter: Fast, Robust Parser Generator for Multi-Language Syntax Trees. <https://github.com/tree-sitter/tree-sitter> (2024)
23. Lewis, P., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 9459–9474 (2020)
24. Joshi, H., Sanchez, J., Sen, K.: RepairLLM: Multi-Stage Program Repair Using Pretrained Models. IEEE Trans. Softw. Eng. **50**(2), 312–329 (2024)
25. Alsaid, M., Nebolisa, C., Abbas, F.: TerraProbe: A Layered-Oracle Framework for Detecting Deceptive Fixes in LLM-Assisted Terraform Security Repair. arXiv preprint arXiv:2606.26590 (2026)
26. Mengistu, M.M., Di Rocco, J., Nguyen, P.T., Di Ruscio, D.: TerraRepair: A Tool-Grounded LLM Agent for Infrastructure-as-Code Repair. arXiv preprint arXiv:2607.11390 (2026)
27. Drosos, G.-P., Sotiropoulos, T., Alexopoulos, G., Mitropoulos, D., Su, Z.: When Your Infrastructure Is a Buggy Program: Understanding Faults in Infrastructure as Code Ecosystems. Proc. ACM Program. Lang. **8**(OOPSLA2), 2490–2520 (2024)